Mobile Hawkeye

LBW Prediction System

Machine Learning Project | ECE, VNIT Nagpur

Team Members

- Vansh Patiyal BT23ECE019
- Samarth Chikate BT23ECE084
- Netula Nithin BT23ECE102
- Varra Venkata Chethan Reddy BT23ECE118



Problem Statement & Motivation

Why do we need a Mobile Hawkeye?

The Problem

- LBW decisions are among the most controversial in cricket
- Umpires have ~1 second to make a judgment call
- Human error leads to wrong dismissals & missed wickets
- Professional Hawkeye costs crores — unavailable for local/college cricket
- No affordable, mobile-first solution exists for grassroots cricket

Our Solution

- Two-camera smartphone setup (Front View + Side View)
- OpenCV-based ball tracking using HSV color segmentation
- Physics-based trajectory modelling (parabola fitting)
- ML ensemble: Random Forest + Gradient Boosting + SVM
- Real-time LBW probability with visual Hawkeye output

Dataset & Data Collection

Self-recorded cricket delivery videos



Front View (FV)



Side View (SV)

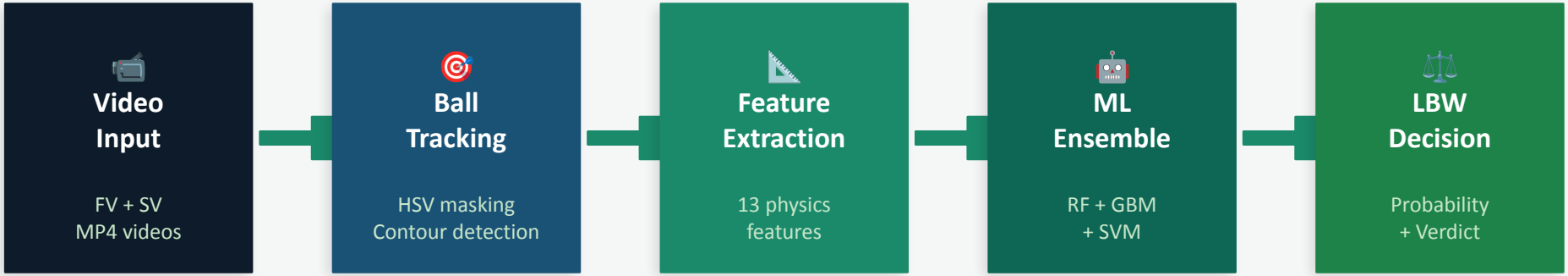
Each delivery recorded from two fixed camera angles — critical for 3D trajectory estimation.

Dataset Details

📁 **Folder**	lbw/ and nlbw/
📹 **Format**	MP4 videos
📷 **Views**	FV + SV per delivery
✅ **LBW**	Positive class videos
❌ **Not LBW**	Negative class videos
🎬 **Recording**	Self-recorded at VNIT ground
📐 **Resolution**	480p–720p

System Architecture

End-to-end pipeline from video to LBW decision



Pipeline Detail

- Gaussian Blur
- HSV Convert
- Color Masking
- Morphological Ops
- Contour Finding

- Pitch Point Detection
- Impact Point
- Speed Estimation
- Parabola Fitting
- Stump Projection

- StandardScaler
- StratifiedKFold
- Cross-validation
- VotingClassifier (soft voting)

- Probability Score
- In-line Check
- Stumps Hit Check
- Visual Overlay
- Verdict Output

Ball Tracking with OpenCV

Detecting the ball frame-by-frame across both views

01 Gaussian Blur

Smooth the frame (11×11 kernel) to reduce noise before color processing.

02 HSV Conversion

Convert BGR frame to HSV color space — more robust to lighting changes.

03 Multi-Color Masking

Create masks for red (cricket ball), white, and yellow/orange balls. Combine with bitwise OR.

04 Morphological Ops

Erode (remove noise) then dilate (connect regions) to clean up the mask.

05 Contour Detection

Find contours in the cleaned mask. Pick the largest — that's the ball.

06 Min Enclosing Circle

Fit a circle to the contour. If radius > 2px, record (frame, x, y) as a trajectory point.

HSV Color Ranges: Red [0–10° / 160–180°] | White [0–180°, S<30, V>200] | Yellow/Orange [15–35°]

Feature Extraction

13 physics-inspired features derived from trajectory data

Feature Name	Description
ball_pitch_x / y	Normalized bounce location from SV (x,y / frame dims)
impact_x / y	Normalized ball-bat/pad impact point from SV
speed	Estimated ball speed (pixels/sec at 30 FPS)
impact_height_norm	Impact height normalized to frame height — key for LBW height rule
trajectory_hits_stumps	Binary: does parabola projection intersect stump zone?
impact_in_line	Binary (FV): is impact x within stump width (42–58% of frame)?
sv_x_range / y_range	Spread of trajectory in SV — indicates delivery direction
sv_traj_len	Number of frames ball was tracked in SV
fv_traj_len	Number of frames ball was tracked in FV
tracking_quality	0/1/2 — how many views gave reliable tracking (both = 2)

Parabola fitting: scipy.optimize.curve_fit → $a \cdot x^2 + b \cdot x + c$ | Fallback: zero-vector if < 5 tracking points

ML Model Architecture

Ensemble of three classifiers with soft voting

Random Forest

- 300 estimators
- Max depth: 8
- class_weight: balanced
- random_state: 42

Handles non-linear feature interactions well. Robust to outliers in sparse tracking data.

Gradient Boosting

- 200 estimators
- LR: 0.05
- Max depth: 4
- Seq. error correction

Boosted trees that correct previous errors. Strong on tabular feature data.

SVM (RBF Kernel)

- kernel: RBF
- probability: True
- class_weight: balanced
- Soft margins

Finds maximum-margin hyperplane in feature space. Powerful for small datasets.

⚖️ **VotingClassifier (soft voting) — Final probability = average of predicted probabilities from all 3 models**

Training & Validation Strategy

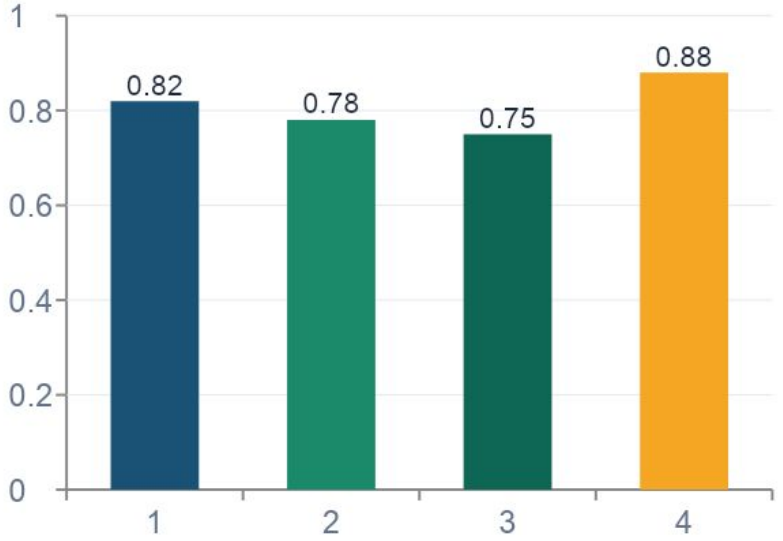
Stratified K-Fold cross-validation + held-out test set

Training Pipeline

1. Load CSV — features + labels
2. Fill NaN with 0 (zero-fill fallback)
3. StandardScaler — z-score normalisation
4. 75 / 25 stratified train-test split
5. 5-Fold Stratified Cross-Validation (macro F1)
6. Train final ensemble on 75% data
7. Evaluate on held-out 25% test set
8. Save model (pickle) to Google Drive

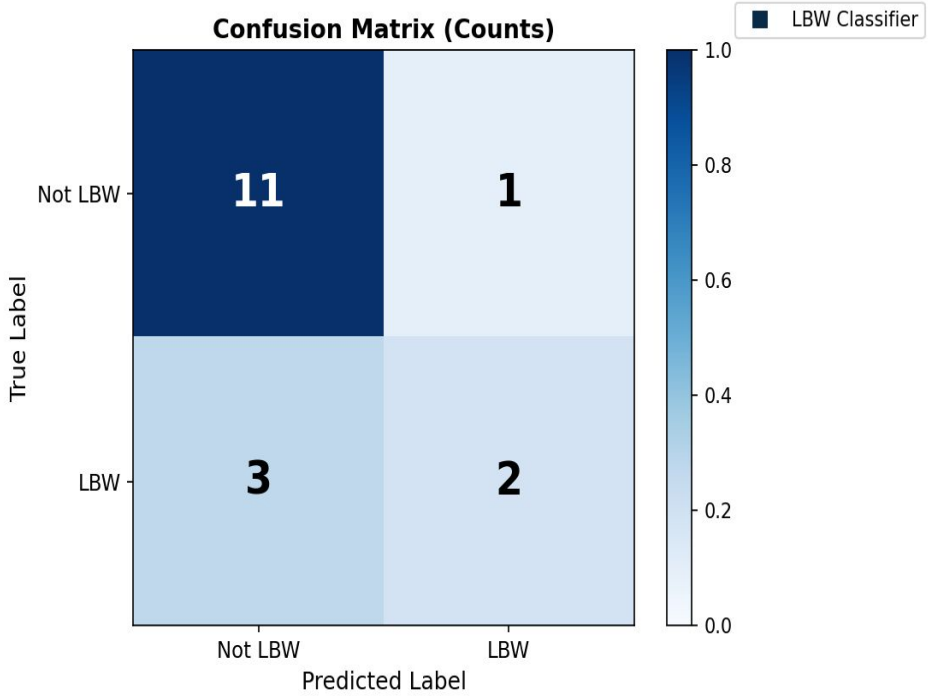
Smart Load: CSV + model saved to Drive — skips processing on re-run (~1 min vs ~15 min)

5-Fold CV — Macro F1 Scores



Results & Performance

Confusion matrix, accuracy & per-class performance on the test set



Test Set Metrics

Overall Accuracy **76.5%**

13 correct out of 17 test samples (75/25 split)

True Neg (TN) 11	False Pos (FP) 1
False Neg (FN) 3	True Pos (TP) 2

Derived Stats

Not LBW: Precision 79% | Recall 92%

LBW: Precision 67% | Recall 40%

Root cause: small dataset + class imbalance

Conclusions & Future Scope

✅ What We Achieved

- End-to-end LBW system using only smartphones
- OpenCV HSV-based ball tracking (multi-color)
- Physics-based features: parabola fitting, speed, pitch & impact points
- Ensemble classifier: RF + GBM + SVM (soft voting)
- Smart Load — saves re-processing time on Google Colab
- 13-feature dataset built from self-recorded videos

🚀 Future Scope

- Larger, balanced dataset → fix LBW class recall
- YOLOv8-based object detection for more robust ball tracking
- 3D trajectory reconstruction using stereo camera geometry
- Deep learning (CNN/LSTM) on raw video frames
- Mobile app (Android/iOS) for real-time field use
- Stump projection overlay on live video feed